

Understanding Large Language Models

Fundamentals, Architecture, and Applied Practice

Table of Contents

1. Executive Summary	1
2. What Is an LLM, Really?	1
3. The Transformer Architecture	1
3.0 Origins: “Attention Is All You Need”	1
3.0.1 Before and After: The LLM Revolution.....	2
3.1 Tokenization & Embeddings.....	3
3.2 Self-Attention	3
3.3 Multi-Head Attention.....	4
3.4 Position, Feed-Forward Layers, and Stability	4
4. How an LLM Is Built: The Three-Stage Training Pipeline	5
5. How an LLM Is Used in Production	5
5.1 Inference Basics	5
5.2 RAG vs. Fine-Tuning	6
5.3 From Single Model to Agentic System	7
5.4 Context Window, Temperature & Sampling	6
5.5 Measuring RAG Quality.....	6
5.6 Production Metrics: Latency, Filtering & Cost.....	7
6. Ecosystem Context	8
6.3 How NVIDIA Accidentally Helped Build the LLM Era	9
7. Applied Summary: Concept-to-Practice Reference.....	8
8. Closing Note	8
9. Challenges & The Road Ahead.....	10

1. Executive Summary

A large language model (LLM) is a neural network trained on vast amounts of text to predict the next word in a sequence. That simple objective, at sufficient scale, produces a system that can write, reason, summarize, translate, and hold conversations — and, when connected to a company's own data, act as the reasoning engine behind production tools.

This document explains how LLMs work end to end: how they represent language, how they are trained, how they are served in production, and how to decide between approaches like retrieval-augmented generation (RAG) and fine-tuning. Concepts are illustrated with concrete, realistic examples of how production LLM systems are typically built, so the ideas translate directly into practical design decisions.

Key takeaway

LLMs are not a single monolithic technology — they are a stack of choices (model, retrieval, orchestration, serving) that must be matched to the problem. Most production quality and cost issues trace back to a mismatch somewhere in that stack, not to the model being "not smart enough."

2. What Is an LLM, Really?

At its core, an LLM is a function that takes in a sequence of text and predicts what comes next, one token at a time.

A **token** is a chunk of text — often a word or part of a word — that the model treats as its basic unit. "Understanding" might become two tokens: "Understand" and "ing". Everything the model does is built on top of this next-token prediction loop, repeated thousands of times to generate a full response.

What makes this powerful is scale: trained on trillions of tokens of text with billions of parameters (the model's internal, learned weights), the model implicitly absorbs grammar, facts, reasoning patterns, and coding conventions — not because it was told the rules, but because predicting text well requires learning them.

3. The Transformer Architecture

3.0 Origins: “Attention Is All You Need”

The transformer was introduced in a 2017 paper titled “Attention Is All You Need,” published by a team of researchers at Google Brain and Google Research. Its central claim was radical at the time: you don't need recurrence (processing words one at a time in order) or convolution to model language well — attention alone, applied in parallel across an entire sequence, is enough. That paper's architecture is,

with relatively few modifications, the same one powering every major LLM today, including GPT, Claude, Gemini, and LLaMA.

Before this paper, state-of-the-art language models were built on recurrent architectures — RNNs and LSTMs — that processed text strictly one word after another, carrying a running “memory” forward. This made them slow to train (each step depended on the previous one, so computation couldn't be parallelized) and prone to losing track of information over long sentences. The transformer's insight — let every word attend to every other word at once, using the self-attention mechanism described in Section 3.2 — removed both bottlenecks simultaneously: training could be parallelized across modern GPU/TPU hardware, and long-range relationships between words no longer decayed with distance.

Why this one paper mattered so much

Removing the sequential bottleneck was what made today's scale possible. Once training could be parallelized, throwing more data and more compute at the model produced predictable, compounding gains (see Section 6.2, Scaling Laws) — which is the direct cause of the jump from narrow NLP tools to general-purpose LLMs.

3.0.1 Before and After: The LLM Revolution

It's worth being explicit about what changed, because the shift wasn't just “better models” — it was a change in what these systems could be used for at all.

	Before Transformers (pre-2017)	After Transformers (LLM era)
Core architecture	RNNs / LSTMs, processed sequentially	Transformers, processed in parallel via attention
Typical scale	Millions of parameters	Billions to trillions of parameters
How it was used	One narrow model per task (translation, sentiment, etc.)	One general model, adapted via prompting, RAG, or fine-tuning
Long-range context	Degraded quickly over long sentences	Maintained across long documents and conversations
Training	Hard to parallelize, slow at scale	Highly parallelizable on GPUs/TPUs
Emergent behavior	Rare — models did what they were explicitly trained for	Reasoning, coding, and instruction-following emerge from scale, not explicit programming

This is the shift that turned language modeling from a collection of specialized NLP tools into a general-purpose reasoning and generation engine — and it's the reason a single model architecture can now sit underneath a PDF question-answering system, a coding assistant, and a customer support agent, with

the difference being data and orchestration around the model rather than a different model for each job.

Every modern LLM (GPT, Claude, Gemini, and LLaMA among them) is built on this same underlying architecture. At a high level, text flows through it in a consistent pipeline: it is tokenized, converted into numerical embeddings, passed through a stack of identical “transformer blocks” that repeatedly refine the model's understanding, and finally converted back into a probability distribution over the next possible token. The diagram below shows that full path.

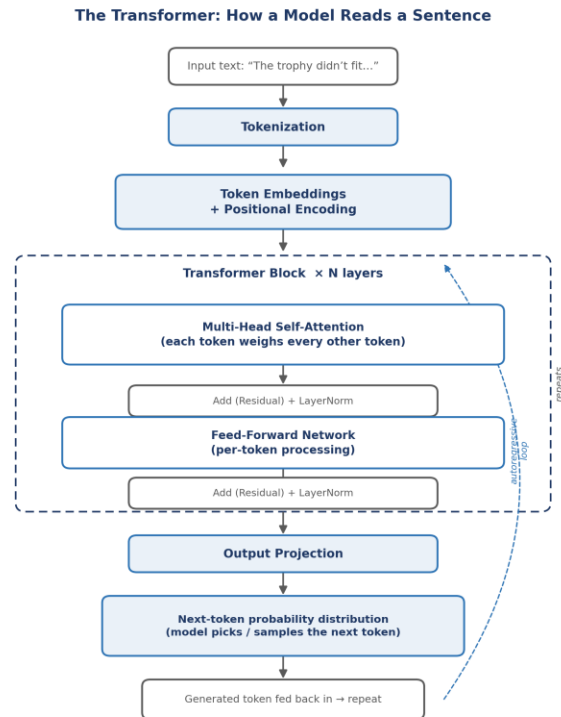


Figure 1 — End-to-end flow of a transformer: from raw text to the next predicted token.

The rest of this section walks through each stage of that diagram.

3.1 Tokenization & Embeddings

Text is first broken into tokens, then each token is converted into a vector — a long list of numbers — called an embedding. This is not arbitrary: the model learns to place words with similar meaning close together in this numerical space. This same idea, applied to whole chunks of text rather than single tokens, is what powers semantic search — it turns a document into vectors so that passages with similar meaning end up near each other, regardless of whether they share exact keywords.

3.2 Self-Attention — "Who Is This Word Talking To?"

Self-attention is the mechanism that lets every word in a sentence look at every other word and decide how much to "pay attention" to it when building its own meaning. In the sentence "The trophy didn't fit in the suitcase because it was too big," attention is what allows the model to correctly link "it" to "trophy" rather than "suitcase" — by weighing the relevance of every other word in context.

Plain-English version

Attention is a dynamic, learned weighting system — for every word, it asks "which other words in this sentence matter most to understanding me?" and blends their information in accordingly.

3.3 Multi-Head Attention

Rather than computing attention once, the model runs several attention computations in parallel ("heads"), each free to specialize — one head might track grammatical structure, another might track topic, another long-range references. Their outputs are combined, giving the model several simultaneous perspectives on the same sentence rather than one flat view.

3.4 Position, Feed-Forward Layers, and Stability

- Positional encoding: since attention has no built-in sense of word order, position information is injected separately so "dog bites man" and "man bites dog" aren't treated identically.
- Feed-forward network (FFN): after attention gathers context, a small neural network processes each token's representation further — this is where a large share of the model's factual and reasoning capacity is stored.
- Residual connections & LayerNorm: bookkeeping mechanisms that keep signal flowing cleanly through dozens of stacked layers so training remains stable at scale.

4. How an LLM Is Built: The Three-Stage Training Pipeline

A model is not trained once — it goes through three distinct stages, each teaching it something different.

1. Pretraining Learn language & world knowledge	2. SFT Learn to follow instructions	3. RLHF Learn to align with human preference
---	---	--

4.1 Pretraining

The model is trained on a huge, broad corpus of text with a single goal: predict the next token. There is no notion of "instructions" yet — this stage is purely about absorbing language patterns, facts, and reasoning structure at scale. This is the most compute-intensive stage, often costing tens of millions of dollars for frontier models.

4.2 Supervised Fine-Tuning (SFT)

The pretrained model is then shown curated examples of instruction-response pairs — a question and a well-written answer — so it learns the format of being an assistant rather than just a text-completion engine. This is the stage that turns a raw language model into something that behaves like a chatbot.

4.3 Reinforcement Learning from Human Feedback (RLHF)

Humans rank multiple model responses to the same prompt from best to worst. A reward model is trained on those rankings, and the LLM is then further trained to produce outputs that the reward model scores highly. This is the stage most responsible for a model feeling helpful, honest, and safe rather than just fluent.

Why leadership should care about this pipeline

Cost, capability, and behavior are set at different stages. If a hosted model's tone or refusal behavior doesn't fit a use case, that is almost always an RLHF-stage property — no amount of prompt engineering fully overrides it. This is one reason model selection matters as much as prompt design.

5. How an LLM Is Used in Production

5.1 Inference Basics

Once trained, generating a response means repeatedly predicting the next token and feeding it back in. Two concepts matter operationally:

- KV cache: intermediate attention calculations are cached per token so the model doesn't recompute from scratch on every new token — this is what makes response streaming fast rather than painfully slow.
- Sampling strategy: how the next token is chosen from the model's probability distribution — deterministic (always the top choice) versus sampled (some controlled randomness) — which trades off consistency against creativity.

5.2 RAG vs. Fine-Tuning — The Decision That Matters Most

This is the single highest-leverage architectural decision in any LLM application. Rather than fine-tuning a model on private or fast-changing data, a RAG system retrieves relevant information at query time — typically via a vector store, often combined with keyword search, and refined with a reranking step — and feeds it into the model's context window alongside the question.

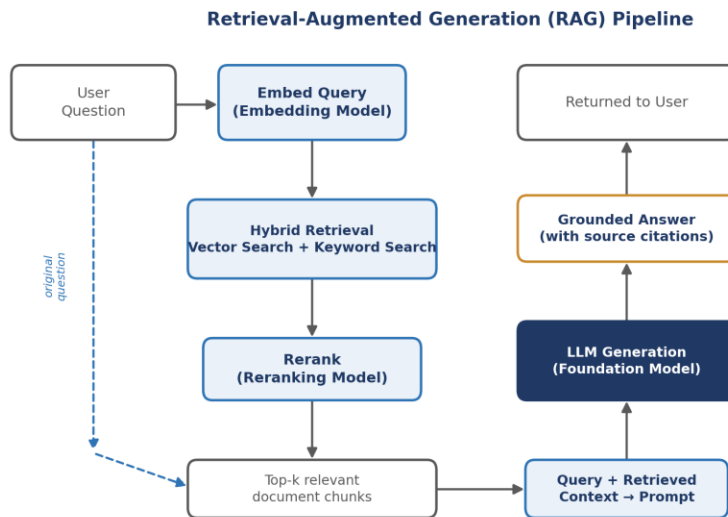


Figure 2 — A typical RAG pipeline: retrieve, rerank, then generate a grounded, citable answer.

Dimension	RAG (Retrieval-Augmented Generation)	Fine-Tuning
What it changes	What the model sees at query time	The model's weights themselves
Best for	Fast-changing or private knowledge (docs, tickets, policies)	Teaching a consistent style, format, or narrow skill
Cost to update	Low — re-index the data	High — retrain or re-tune the model
Transparency	High — answers can cite retrieved source chunks	Low — knowledge is baked in, harder to audit

Dimension	RAG (Retrieval-Augmented Generation)	Fine-Tuning
Typical use case	Document Q&A, search, support assistants	Style transfer, structured output, narrow specialist tasks

In practice, most production systems lean on RAG first because it is cheaper to iterate on, keeps data auditable, and avoids retraining every time source documents change. Fine-tuning becomes worthwhile mainly when the need is a specific behavior or format the model can't reliably be prompted into, not new factual knowledge.

5.3 From Single Model to Agentic System

A single LLM call answers a question. An agentic system chains multiple LLM calls together with tools, memory, and decision points, allowing the system to break a task into steps, call external tools (e.g., search, messaging platforms, internal APIs), and adapt based on intermediate results rather than producing one static output.

5.4 Context Window, Temperature & Sampling

Beyond training and retrieval, a few inference-time settings directly shape what a model can see and how it chooses its words. These are the levers engineers actually tune day to day.

Context window: the maximum number of tokens the model can process in a single call, typically ranging from around 4K tokens in smaller models to 200K or more in frontier models. Everything the model can reason about — the conversation, retrieved documents, instructions — must fit inside this window. A larger window doesn't guarantee better use of that space; models can still lose track of details buried in the middle of a very long context.

Temperature: controls how much randomness is applied when choosing the next token. A low temperature (around 0.1) makes output deterministic and repeatable, favoring the single most likely token every time — useful for factual or structured tasks. A high temperature (1.0 or above) allows less-likely tokens to be chosen more often, producing more varied, creative, but less predictable output.

Top-p (nucleus sampling): a complementary control that samples only from the smallest set of tokens whose combined probability exceeds a threshold p , rather than considering every possible token. This trims away the long tail of very unlikely tokens before randomness is applied, which keeps high-temperature output creative without letting it wander into incoherence.

Practical tip

Temperature and top-p are usually tuned together, not separately: a common production default is a moderate temperature (0.2–0.7) paired with a top-p around 0.9, adjusted per task — lower for factual Q&A, higher for creative writing or brainstorming.

5.5 Measuring RAG Quality: Context Relevance, Groundedness & Answer Relevance

A RAG system can fail in ways that “the model is wrong” doesn’t fully explain — often the model reasoned correctly over bad or incomplete context. These three metrics separate out where a failure actually happened.

Context relevance measures whether the chunks retrieved for a query actually contain the information needed to answer it. It degrades due to vocabulary mismatch between the query and source wording, chunk boundaries that cut a passage mid-thought, semantic drift in very large chunks, or stale duplicate documents competing with the current version in the index. It’s controlled with hybrid search (combining vector similarity with keyword matching), a reranking step over a wider candidate set, and chunk sizes tuned per document type.

Groundedness (faithfulness) measures whether the model’s answer is actually supported by the retrieved context, rather than the model filling gaps from its own general knowledge, which may be outdated or invented. A response can be fluent and even factually correct in general and still be ungrounded, meaning it wasn’t really derived from what was retrieved. This is improved by prompting the model to answer only from the provided context and say so when context is insufficient, attaching citations to each claim, and running an automated check that compares the answer against the retrieved chunks.

Answer relevance is a separate axis again: an answer can be perfectly grounded in the retrieved text and still fail to address what the user actually asked, for example answering a related but different question. This is typically measured by comparing the semantic similarity of the generated answer back to the original query.

Why these metrics matter together

A RAG system can score well on one axis and fail on another — high context relevance with low groundedness usually means the model is ignoring good context; high groundedness with low answer relevance usually means retrieval found the wrong-but-related document. Debugging RAG requires measuring these separately, not just eyeballing whether the final answer “looks right.”

5.6 Production Metrics: Latency, Metadata Filtering & Cost-Effectiveness

Once a RAG or agentic system is live, three operational concerns determine whether it’s actually usable at scale.

Latency in production is a sum of several stages, each with its own optimization levers. A common mistake is optimizing model inference speed while ignoring that retrieval and reranking can together account for more total latency than generation itself, especially with large indexes.

Stage	What drives latency	Typical lever
Retrieval	Vector store size, index type	Approximate nearest-neighbor indexes (HNSW, IVF)
Reranking	Number of candidates reranked	Rerank fewer, higher-quality candidates
Generation	Model size, output length, KV cache	Smaller model for simple queries, streaming output
Orchestration	Number of chained tool/LLM calls	Parallelize independent calls, cache repeat queries

Metadata filtering success refers to whether filters applied before or during retrieval — by document date, department, access permissions, or type — correctly narrow the search space without silently excluding the right document. Common failure points are inconsistent tagging at ingestion time, overly strict filters that reject valid near-matches, and filters that aren't updated when access permissions change. This is worth monitoring separately from context relevance: a retrieval miss caused by a bad filter looks identical to one caused by bad embeddings, but the fix is completely different.

Cost-effectiveness in a RAG system comes from three places: indexing cost (usually one-time or incremental), retrieval infrastructure cost (scales with data volume), and generation cost (usually the largest and most controllable piece, scaling with context size and output length). The main levers are:

- Retrieve fewer, better chunks — reranking allows a smaller final context with the same quality
- Cache frequent queries and their retrieved context
- Route simple queries to smaller, cheaper models and reserve larger models for complex reasoning
- Compress or summarize long retrieved context before passing it to the model

6. Ecosystem Context

6.0 Key Components of the LLM Ecosystem

A production LLM application is rarely just “the model.” A typical request flows from the user through an application layer, out to an inference provider hosting the foundation model, and — for RAG systems — in parallel through a retrieval path that turns a knowledge source into searchable vectors and back into relevant context. The table below maps each moving part to its role.

Component	Role
Foundation Model (LLM)	Understands and generates text (e.g., GPT-4, Claude, Llama, Gemini).
Inference Provider	Hosts and serves the LLM via APIs (e.g., Anthropic, OpenAI, Groq, Together AI).
Embedding Model	Converts text into semantic vectors (e.g., MiniLM, BGE, E5).
Vector Database	Stores and retrieves vector embeddings (e.g., Chroma, Pinecone, FAISS).
Retriever	Finds the most relevant information via keyword, semantic, or hybrid search.
Prompt	Combines the user's query with retrieved context before it reaches the LLM.

6.1 Mixture of Experts (MoE)

Instead of every parameter in the model activating for every input, MoE architectures route each token through only a subset of specialized sub-networks ("experts"). This allows models to have a very large total parameter count while keeping the actual compute per token much lower — more capacity without a proportional cost increase.

6.2 Scaling Laws

Empirically, model performance improves in a predictable way as model size, dataset size, and compute grow together, provided they scale in balance with each other. This is why frontier labs invest so heavily in compute and data pipelines — the returns to scale are consistent, not speculative — though gains do flatten and require proportionally larger investment at the frontier.

6.3 How NVIDIA Accidentally Helped Build the LLM Era

One of the more genuinely accidental turning points in modern AI history is that a company built entirely for video game graphics ended up supplying the compute that made deep learning — and eventually LLMs — possible.

The setup. NVIDIA's GPUs were designed for one job: rendering millions of pixels in parallel for video games. That requirement — doing the same simple math operation across huge amounts of data simultaneously — happens to be mathematically identical to what neural networks need: massive parallel matrix multiplication. Nobody designed GPUs with AI in mind; the overlap was structural coincidence. The bridge came in 2006, when NVIDIA released CUDA, a toolkit that let developers program GPUs for general-purpose computation instead of only graphics.

The moment it mattered: AlexNet, 2012. By 2011, Google's Brain project had gotten neural networks to recognize cats in YouTube videos, but doing so required 2,000 CPUs running in one of Google's data centers — infrastructure only a handful of companies on Earth could afford. That same year, an NVIDIA researcher teamed up with Andrew Ng's lab at Stanford and found that just 12 NVIDIA GPUs could match the deep-learning performance of those 2,000 CPUs. Then, in 2012, three University of Toronto researchers — Alex Krizhevsky, Ilya Sutskever, and Geoffrey Hinton — built an image-recognition network, later known as AlexNet, using just two consumer gaming GPUs running in SLI, a setup normally sold to gamers wanting smoother frame rates. AlexNet entered that year's ImageNet competition and won by a massive margin over every other approach, most of which were still running on CPUs. This result is widely regarded as the “big bang” moment that proved deep learning at scale was practical — and that it was practical specifically because of gaming hardware repurposed for it.

Why it matters for LLMs today. AlexNet proved the economics of deep learning: research that once required data-center-scale budgets could now be done with a couple of consumer graphics cards, opening the field to university labs and startups rather than a handful of tech giants. It also set the template every LLM still depends on — the transformer architecture's core advantage (Section 3.0) is that it parallelizes training, which only pays off because GPUs, unlike CPUs, can run thousands of matrix operations at once. Following AlexNet, NVIDIA's hardware line evolved specifically to serve deep learning workloads; the NVIDIA A100 GPU is what made large-scale distributed training of today's large models practically feasible, and leading LLMs today are typically trained on GPU clusters exceeding 10,000 GPUs — a direct descendant of the two-GPU AlexNet setup, scaled up roughly five-thousand-fold.

The accidental part

NVIDIA didn't build GPUs for AI — it built them to make video games look better. The company's pivot to becoming the backbone of the AI industry happened only after outside researchers proved the hardware could do the job, a reminder that some of the most consequential infrastructure decisions in tech history were downstream of someone using a tool for a purpose it wasn't built for.

7. Applied Summary: Concept-to-Practice Reference

Concept	How it typically shows up in a production system
Embeddings	An embedding model turning documents or chunks into searchable vectors
Retrieval	A vector store, often paired with keyword search and a reranking step
Inference	A hosted or self-served foundation model generating responses
Orchestration	A multi-step agent framework chaining calls, tools, and decision points
Production architecture	A backend API layer serving a lightweight frontend for multi-user access

8. Closing Note

Well-built LLM systems don't treat the model as a black box — they combine deliberate architectural choices at every layer: how text is embedded, how relevant context is retrieved, which model generates the final answer, and how multiple steps are orchestrated into a coherent workflow. Understanding these fundamentals is what allows those choices to be made deliberately rather than by trial and error, and it's what keeps a roadmap moving efficiently as a project scales.

9. Challenges & The Road Ahead

No production LLM deployment is free of trade-offs. Being explicit about the current limits — and where the field is headed — is part of using this technology responsibly.

9.1 Current Challenges

- **Hallucinations:** models generate plausible but incorrect information, because generation is fundamentally a statistical prediction of the next token, not a database lookup. This is inherent to how LLMs work, not a bug that gets fully patched away.
- **Bias & safety:** training data reflects the biases present in the text it was drawn from, which requires deliberate alignment work (see Section 4.3, RLHF) to correct for at the model level, and careful system design to correct for at the application level.
- **Cost & latency:** large models demand significant compute and energy; Section 5.6 covers the specific levers used to manage this in production.
- **Context limits:** very long documents still challenge even frontier models — a large context window (Section 5.4) does not guarantee the model weighs every part of it equally well.

9.2 What's Next

Alongside these challenges, four directions are shaping where LLMs, and the systems built around them, are headed next.

Direction	What it means
Multimodal	Text, image, audio, and video understanding unified in a single model rather than separate systems.
Agentic AI	Autonomous task execution and planning, chaining tools and decisions with less human orchestration.
Efficiency	Smaller, faster, cheaper models that preserve most of the capability of much larger ones.
Reasoning	Deeper logical and mathematical capability, including more reliable multi-step problem solving.